VIETNAM–KOREA UNIVERSITY OF INFORMATION AND COMMUNICATION TECHNOLOGY (VKU)
Faculty of Computer Science

PROJECT PROPOSAL & INTERACTIVE WIREFRAME PROTOTYPE

NGÔN SẮC VIỆT

Nền tảng AI dịch và bảo tồn ngôn ngữ dân tộc Việt Nam
AI Translation and Heritage Preservation Platform for Vietnamese Ethnic Minority Languages

Course	Cross-Platform Mobile App Development
Project Title	Ngôn Sắc Việt – Nền tảng AI dịch và bảo tồn ngôn ngữ dân tộc Việt Nam (Ngon Sac Viet – AI Translation and Heritage Preservation Platform for Vietnamese Ethnic Minority Languages)
Team Name	Ngôn Sắc Việt
Academic Term	Semester 1 (2026 – 2027)
Deadline	Week 4

Team Members & Responsibilities

The project has three parts: the AI models, the backend and the Flutter app. Each member leads one part, and all four members build the Flutter app together, because it is the product the course evaluates and the place where the three parts meet.

No.	Full Name	Student ID	Main responsibility	Part in the Flutter app
1	Bùi Thị Lệ Thắm (leader)	23IT253	Backend in Python (FastAPI) on Supabase PostgreSQL. Coordinates the team's work and the user flows (sign-up, roles, contribution and review), and designs an architecture that can be extended to more languages and features later.	Completes the model and API layer of the app: generated data models, the HTTP client that mirrors every backend endpoint, token handling, error mapping, and the native integrations (on-device OCR, secure storage, file picker).
2	Nguyễn Minh Duy	23IT038	AI model in Python, together with Nguyên. Researches previous work on the five ethnic languages (datasets, models, related sources), builds the text translation model first, then the speech models.	Sets up the base code with a clean architecture (data, domain, ui), dependency injection with Riverpod, build-time configuration, and the start-up flow that loads the bundled catalogue into the database.
3	Lương Thế Nguyên	23IT183	AI model in Python, together with Duy. Collects and cleans parallel text and audio, maintains the training-data pipeline, and works on speech recognition and OCR after the text model.	Builds the local database and synchronisation: SQLite storage, the local store, the offline contribution queue, catalogue sources (API or Supabase) and the sync use case with its tests.
4	Trần Đình Nhứt	23IT203	Frontend in Flutter: localization, light and dark mode, go_router, model generation and current Flutter libraries.	Builds the UI from the Figma design, connects the screens to the data, and finishes and integrates all previous parts into the final app.

Order of work on the Flutter app. Each step starts from the code left by the previous one, so every member knows the whole app and not only one folder:

1. Nguyễn Minh Duy creates the base code: folder structure following clean architecture, Riverpod providers and overrides, configuration through --dart-define (API address, Supabase keys), and the code that opens the local database and fills it with the bundled catalogue on first launch.
2. Bùi Thị Lệ Thắm completes the skeleton of models and API: freezed and json_serializable models matching the backend, the dio client with the token interceptor, conversion of server errors into readable messages, and the native parts (Google ML Kit OCR, flutter_secure_storage, file_picker) with separate implementations for Android/iOS and web.
3. Lương Thế Nguyên builds the local database and sync: the SQLite document store, LocalStore, the queue of contributions written offline, and SyncUseCase, which downloads the catalogue and uploads pending items when the network is available.
4. Trần Đình Nhứt builds the interface from the design (Translate, Heritage, Learn, Contribute, Settings, Login), adds localization and themes, connects the screens to the data from the previous steps, and completes the app end to end.

Every member also tests modules written by someone else before they are merged.

---

1. PROJECT MOTIVATION & PROBLEM STATEMENT

1.1. Context & Practical Problem Space

Vietnam has 54 officially recognised ethnic groups, and most of their languages are low-resource: there is little digital text, almost no public parallel corpus, and mainstream translation tools either do not support them or support a different written standard. For H'Mông, online tools generally follow the overseas RPA spelling rather than the Latin orthography taught in Vietnam, and Cham written in the traditional Akhar Thrah script is not handled at all.

This gap shows up in everyday situations. A nurse posted to a commune health station in Gia Lai cannot read a short note a Jrai patient brings in. A teacher in Hà Giang has no quick way to check a H'Mông phrase in a parent's message. Ethnic-minority students who move to the city, including students at VKU, use Vietnamese every day and slowly lose their mother tongue because there is no mobile tool to look up, translate and practise it. At the same time, epics, customary law, greetings and songs are scattered across books, radio archives and personal phones, and are rarely linked to a translation anyone has checked.

Ngôn Sắc Việt is a mobile application whose main purpose is translation between ethnic minority languages, Vietnamese and English, through three inputs: typed text, a photo of printed text (OCR) and recorded speech. Three supporting modules sit around this core and feed it with data: community contribution and review of sentence pairs, a heritage archive for each people, and short lessons with quizzes. Every approved community correction becomes training data for the next version of the model, so the translation quality is expected to improve as the app is used.

Target languages for the first release

We start with five languages that each have a sizeable speaker community and at least one usable data source. Vietnamese and English serve as pivot languages. The table summarises what data exists for each language and how that affects our plan.

Language	ISO 639-3	Family	Script	Main region	Available data and plan
Tày	tyz	Tai-Kadai	Latin (based on Quốc ngữ)	Cao Bằng, Lạng Sơn, Bắc Kạn	Public Tày–Vietnamese parallel text. First language to fine-tune.
Ê Đê (Rade)	rad	Austronesian, Chamic	Latin	Đắk Lắk	Public Ê Đê–Vietnamese corpus. Main data source in phase one.
Chăm (Eastern)	cjm	Austronesian, Chamic	Akhar Thrah and Latin	Ninh Thuận, Bình Thuận	Very little parallel text. Transfer from Ê Đê and Jrai; Cham-script OCR is a separate task.
H'Mông	hmn	Hmong-Mien	Latin (Vietnamese orthography)	Hà Giang, Lào Cai, Điện Biên	VOV recordings with H'Mông text and Vietnamese. First language for speech recognition.
Jrai	jra	Austronesian, Chamic	Latin	Gia Lai, Kon Tum	A Jrai–Vietnamese SMT system (2012) and dictionaries, few public sentence pairs. Transfer from Ê Đê.

1.2. Target User Personas

Primary persona: H'Linh Niê, 20, an Ê Đê student at VKU from Buôn Ma Thuột. She wants to keep her Ê Đê fluent, translate family messages and village documents into Vietnamese, and share Ê Đê stories with classmates. Her problems are that no translator supports Ê Đê, the bilingual dictionaries are at home rather than on her phone, and when she knows a better translation she has nowhere to record it for others. She uses a mid-range Android phone and has unstable 4G when she goes back to her village.

Secondary persona: Nguyễn Văn Hùng, 28, a Kinh nurse at a commune health station in Gia Lai. He needs to understand short Jrai phrases and read notes patients bring, often without a network connection. He cannot type Jrai words he does not know how to spell, so camera and voice input matter more to him than the keyboard.

Supporting persona: a community reviewer, usually a bilingual teacher or village elder, who checks submitted sentence pairs so that only verified data reaches the public dictionary and the training set.

Typical user story

1. Hùng opens the app. The language list, peoples and lessons are already on the phone because they were bundled with the app and refreshed at the last sync.
2. He takes a photo of a patient's printed note. Google ML Kit recognises the Latin text on the device, and the text is placed in the input box so he can correct it before translating.
3. He selects Jrai to Vietnamese and taps Translate. The result shows the translated sentence together with the route taken (direct, or through Vietnamese) and any warning. If no reviewed model covers this direction yet, the app says so instead of showing a guessed sentence.

4. Later, H'Linh adds a better translation for a phrase in the Contribute tab. It is saved on her phone with a unique client ID and uploaded at the next successful sync, even if she was offline when she wrote it.
5. A reviewer approves the pair. It then appears for all users, and the AI team exports it as a reviewed training pair for the next fine-tuning round.

Other user stories we will support:

- As a learner, I want short lessons and a quiz on basic Tày greetings so that I can practise for ten minutes a day.
- As a visitor, I want to read where the Chăm language comes from and see photos and short videos so that I understand the culture behind the words.
- As a reviewer, I want a queue of pending contributions that I can approve or reject with a note so that only verified data is published.

---

2. SYSTEM SCOPE & FUNCTIONAL SPECIFICATIONS

Priorities follow the MoSCoW method. Since translation is the reason the app exists, every translation input is Must-Have or Should-Have. The heritage and learning modules reuse the same catalogue and are scheduled after the translation loop works end to end.

No.	Feature Category	Concrete Feature	Priority	Business Logic & Scope
1	Authentication & Profile	Email and password sign-up and login, token refresh, logout, profile; Google sign-in through Supabase Auth; roles member and reviewer	Must-Have	Short-lived JWT access token and a refresh token stored only as a SHA-256 hash and replaced on every refresh. Passwords hashed with Argon2. Guests can translate; contributing requires an account.
2	Core Business Logic: text translation	Ethnic language to and from Vietnamese and English; automatic detection of the source language; swap and copy	Must-Have	The router uses a direct pair if one exists, otherwise goes through Vietnamese (for example Ê Đê to Vietnamese to English). The reviewed glossary is checked before the neural model. If neither covers the direction, the API returns model_not_ready.
3	Artificial Intelligence: machine translation	mT5-small fine-tuned for ethnic languages and Vietnamese; NLLB-200-distilled-600M for Vietnamese and English	Must-Have	Input format "src tgt: sentence", sentences up to 128 tokens. Trained only on reviewed pairs. Evaluated per language on a held-out test set.
4	Artificial Intelligence: photo translation (OCR)	Pick or take a photo, recognise the text, edit, translate	Must-Have	Google ML Kit on Android and iOS for Latin script (Tày, Ê Đê, H'Mông, Jrai). PaddleOCR on the server as fallback. Akhar Thrah needs its own model and is planned for phase two. Images up to 8 MB.
5	Artificial Intelligence: voice translation (ASR)	Record speech, show the transcript, translate	Should-Have	Whisper fine-tuned on H'Mông VOV recordings first, other languages as audio is collected. Audio up to 20 MB. The transcript is editable before translation.

No.	Feature Category	Concrete Feature	Priority	Business Logic & Scope
6	Community contribution	Submit a new or corrected sentence pair; list of my contributions with status	Must-Have	Status goes from pending to approved or rejected. A repeated upload with the same client ID does not create a duplicate. Same source and target language, or unknown codes, are rejected.
7	Community review and data export	Review queue, approve or reject with a note, export approved pairs as CSV	Must-Have	Only reviewers can review and export. Only approved pairs are published and used for training.
8	Heritage archive	Profile of each people (region, language family, origin of the language), articles, photos, short videos	Should-Have	Bilingual content in PostgreSQL, media in Supabase Storage. Media is attached to reviewed text only.
9	Language learning and quiz	Short lessons with verified phrases; multiple-choice quiz with explanations	Should-Have	The quiz is scored on the server. Answer keys are not readable with the public key and are never stored on the phone.
10	Offline use	Local copy of the catalogue, heritage and lessons; offline contribution queue; automatic sync	Should-Have	SQLite on the device, catalogue bundled with the app for the first launch, sync on start-up and on demand.
11	Localization and theme	Vietnamese and English interface; light, dark and system theme	Must-Have	Flutter gen-l10n with ARB files, Material 3. Choices are saved on the device.
12	Translation history	Recent translations of the user	Could-Have	Stored in translation_logs (mode, route, engine), also used to see which directions are requested most.

2.1. How a translation request is processed

Because translation is the core of the project, we specify its behaviour in more detail than the other modules.

1. Input. Text is typed, read from a photo by OCR, or transcribed from audio by ASR. For OCR and ASR the recognised text is shown first so the user can fix mistakes; translating a wrong transcript only multiplies the error.
2. Normalisation. Extra whitespace is removed and the text is lower-cased for glossary lookup. Diacritics are kept because they carry meaning in all five languages.
3. Language detection. Text in the Cham Unicode block (U+AA00 to U+AA5F) is identified as Cham directly. The Latin-script ethnic languages share most letters with Vietnamese, so until a trained language-ID model is available the app asks the user to choose the source language instead of guessing.
4. Route planning. If a reviewed glossary or a model covers the pair directly, it is used. Otherwise the request is split into two steps through Vietnamese. If one of the steps is not covered, the request stops with model_not_ready.

5. Translation. Each step tries the reviewed glossary first and then the neural model. The response lists every step with the engine used, so the user and the team can see where an answer came from.
6. Logging. The gateway stores mode, languages, route and engine in translation_logs. These logs show which directions users need and help us decide what data to collect next.

2.2. Evaluation plan for the translation model

- For each language, 10% of the reviewed pairs are held out as a test set and never used for training.
- Automatic metrics: SacreBLEU and chrF. chrF is reported alongside BLEU because it is more stable for short sentences and rich morphology.
- Baseline: the reviewed glossary alone and the untuned base model. A fine-tuned model is only released if it beats both on chrF.
- Human check: 50 sentences per language rated by a native speaker for adequacy and fluency on a 1–5 scale.
- Language ID, when trained: accuracy and macro-F1 on short sentences across the five languages plus Vietnamese and English.

2.3. Non-functional requirements

- Response time: under 2 seconds on the glossary path and under 5 seconds on the neural path (95th percentile, CPU server). On-device OCR under 1 second for a normal photo.
- No invented output: unsupported directions return a clear, localized message.
- Security: HTTPS only, Argon2 password hashing, refresh-token rotation, Row-Level Security so the public key can only read the public catalogue. Contributions are written only through the authenticated API.
- Extensibility: a new language is added through the shared catalogue file and reviewed data, without changing the app code.
- Accessibility: readable contrast in both themes, scalable text, large touch targets for camera and microphone.

2.4. Out of scope this semester

- Text-to-speech, speech-to-speech and translation of long documents. The app targets short sentences and phrases.
- Training a foundation model from scratch. We fine-tune existing multilingual models.
- OCR of images without text, and any payment features.

2.5. Interactive wireframe: screens and navigation

The Figma prototype contains the screens below. The route column uses the same paths as the go_router configuration in the app, so each frame in Figma corresponds to one route in code. The four main screens are tabs in a bottom navigation bar.

No.	Screen	Route	Main components and interactions
1	Translate (tab 1)	/translate	Source and target language selectors, text box, "Read from photo" button, Translate button, result with route and warnings.
2	Voice input	/translate/voice (planned)	Hold to record, editable transcript, translate.
3	Heritage list (tab 2)	/heritage	Card for each people with name and region.
4	Heritage detail	/heritage/:slug	Region, language family, origin of the language, articles, photos and short videos.
5	Lessons (tab 3)	/learn	Lessons grouped by language.
6	Lesson and quiz	/learn/:lessonId	Lesson summary, verified phrases, multiple-choice questions, score and explanation after submitting.
7	Contribute (tab 4)	/contribute	Language, source sentence, Vietnamese sentence, note; list of my contributions with status (pending shown even when offline).
8	Review queue	/contribute/review (planned, reviewers only)	Pending pairs with approve and reject buttons and a note.
9	Settings	/settings	Light, dark or system theme; Vietnamese or English interface.
10	Sign in / Register	/login	Email and password, Google sign-in, continue as guest.

3. TECH STACK & INITIAL ARCHITECTURE

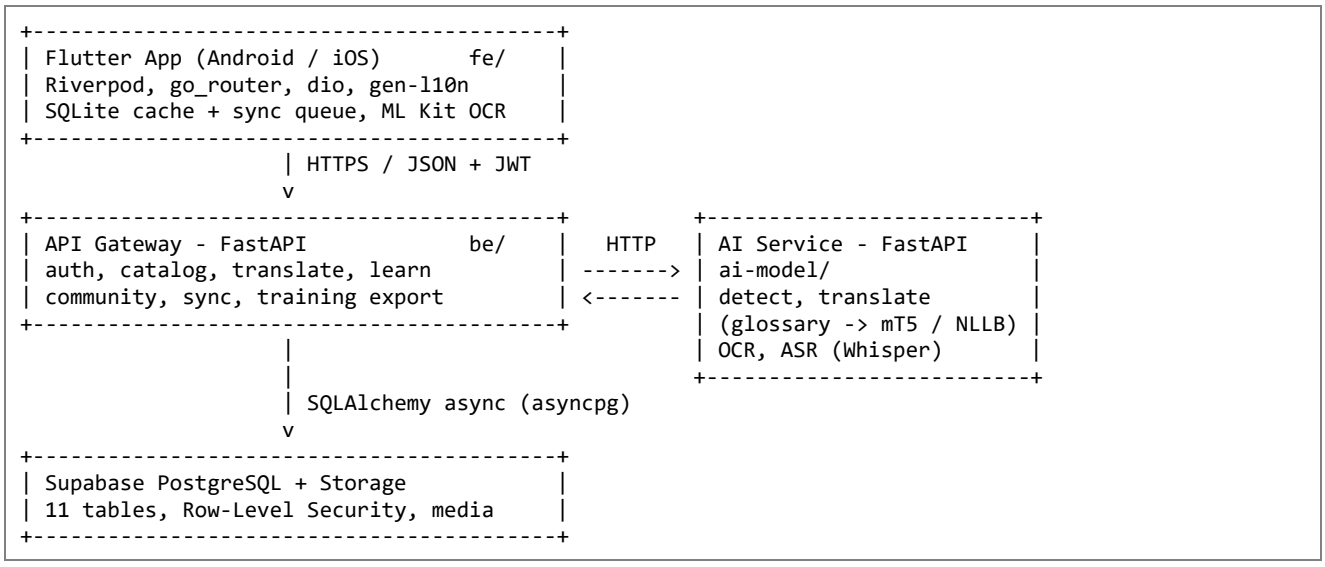
Framework: Flutter 3.x (Dart 3.8), Material 3. State management: Riverpod 3. Backend: FastAPI (Python 3.11) on Supabase PostgreSQL. AI engine: Hugging Face Transformers and PyTorch (mT5-small, NLLB-200-distilled-600M, Whisper), with Google ML Kit for on-device OCR.

Layer	Technology	Reason for the choice
Mobile framework	Flutter 3.x, Material 3	One codebase for Android, iOS and a web demo; renders Vietnamese and ethnic Latin diacritics correctly.
State management	Riverpod 3	Providers can be overridden in tests, which lets us run the same screens with offline or online data sources.
Navigation	go_router 17 (StatefulShellRoute)	Each tab keeps its own navigation stack; deep links such as /heritage/ede work directly.
Models and HTTP	freezed, json_serializable, dio	Generated immutable models; one interceptor adds the access token to every request.
Localization and theme	gen-l10n with ARB files, ThemeMode	Vietnamese and English strings; theme and language saved with shared_preferences.
Local storage	sqflite, flutter_secure_storage	Offline catalogue and contribution queue in SQLite; tokens in the secure keystore.
Backend	FastAPI, SQLAlchemy 2 (async), asyncpg, Pydantic v2	Async REST API with typed validation and automatic OpenAPI documentation.

Layer	Technology	Reason for the choice
Database and storage	Supabase PostgreSQL, Row-Level Security, Storage	Managed Postgres; the public key can read only the public catalogue; storage for heritage media.
Authentication	PyJWT (HS256), pwdlib (Argon2), Supabase Auth for Google	Short-lived access tokens and rotating hashed refresh tokens.
AI	Transformers, PyTorch, mT5-small, NLLB-200-distilled-600M, Whisper, Google ML Kit, PaddleOCR	Fine-tuning existing multilingual models is realistic for low-resource languages within one semester.
Evaluation	SacreBLEU, chrF, accuracy and F1	Standard and reproducible metrics for translation and language ID.
Tooling	Docker, GitHub, pytest, flutter_test	Same environment for all members; tests run before each merge.

3.1. Initial system architecture

The system has three deployable parts. The app only talks to the API gateway. The gateway owns users, data and logs, and calls the AI service over HTTP, so a model can be replaced or moved to a GPU machine without any change in the app. The app may also read the public catalogue directly from Supabase with the anonymous key; Row-Level Security limits that key to read-only public tables.



The database has eleven tables: users, refresh_tokens, languages, ethnic_groups, heritage_items, lessons, phrases, quiz_questions, quiz_choices, translation_logs and contributions. The list of languages and peoples is defined once in shared/languages.json and seeded into the database and into the app's bundled catalogue, so all three parts use the same language codes.

3.2. AI development plan

Phase	Weeks	Output	Owner
Research	1 – 3	Survey of existing datasets, models and tools for the five languages; licence check; first reviewed glossary.	Duy, Nguyên

Phase	Weeks	Output	Owner
Text translation model	4 – 6	Reviewed parallel set (500–800 pairs to start), mT5-small fine-tuned, NLLB for Vietnamese–English, BLEU and chrF report.	Duy (lead), Nguyên
Language identification	5 – 6	Short-sentence classifier for the five languages plus Vietnamese and English.	Duy, Nguyên
Speech and OCR	6 – 8	Whisper fine-tuned on H'Mông VOV data; OCR pipeline with ML Kit and PaddleOCR; study of Cham-script OCR.	Nguyên (lead), Duy
Data loop	7 – 9	Approved community pairs exported, validated and used for the next fine-tuning round.	Whole team

3.3. Risks and mitigation

Risk	Impact	Mitigation
Very little parallel data for Chăm and Jrai	Poor translation quality	Transfer from the related Chamic language Ê Đê; community contributions; explicit model_not_ready instead of guessing.
H'Mông spelling in Vietnam differs from the overseas RPA	Wrong spelling in training data and output	Keep only the Vietnamese orthography; do not mix in Hmong–Thai or RPA data.
Model produces fluent but wrong sentences	Users lose trust; cultural harm	Reviewed glossary first, training on reviewed data only, engine and warnings shown with every result.
Weak network in highland areas	App not usable in the field	Bundled catalogue, SQLite cache, on-device OCR, contribution queue.
Limited GPU time	Slow training cycles	Small models (mT5-small, distilled NLLB), short sentences, free cloud GPU sessions.

3.4. Links

Figma prototype URL: *[https://www.figma.com/file/xxx/your-link]*

GitHub repository: *[https://github.com/username/your-repo-name]*. Folders: fe (Flutter app), be (FastAPI and Supabase migrations), ai-model (AI service and training scripts), shared (language catalogue).